Lab Assignment 13.2

Program	**: B. Tech (CSE)**
Course Title	**: AI Assisted Coding**
Course Code	**: 23CS002PC304**
Semester	**: III**
Academic Session	**: 2025-2026**
Name of Student	**: Sruthi**
Enrollment No.	**: 2403A51L10**
Batch No.	**: 51**
Date	**: 10-03-2026**

Task 1: Refactoring – Eliminating Repetitive Logic

Sample code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Prompt: Restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design and provide docstrings

Code:

```
def calculate_area(length, width):
    """Calculate the area of a rectangle.
    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.
    Returns:
        float: The area of the rectangle.
    """
    return length * width
def calculate_perimeter(length, width):
    """Calculate the perimeter of a rectangle.
    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.
    Returns:
        float: The perimeter of the rectangle.
    """
    return 2 * (length + width)
print("Area:", calculate_area(8, 4))
print("Perimeter:", calculate_perimeter(8, 4))
print("Area:", calculate_area(12, 6))
print("Perimeter:", calculate_perimeter(12, 6))
```

Output:

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Explanation:

The sample code had repeated calculations for area and perimeter of rectangles with different dimensions. By defining two functions, `calculate_area` and `calculate perimeter`, we can reuse the logic for calculating area and perimeter for any given length and width.

Each function takes the length and width as parameters and returns the calculated area or perimeter.

This design is more maintainable and reusable, as we can easily call these functions with different dimensions without duplicating code. Additionally, the docstrings provide clear explanations of what each function does, its parameters, and its return value, enhancing code readability and maintainability.

Task 2: Refactoring – Modularizing Inline Calculations

Sample input code:

```python
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

Prompt: Refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Code:

```python
def calculate_final_amount(amount):
    """Calculate the final amount after applying a 10% discount.
    Args:
        amount (float): The original amount before discount.
    Returns:
        float: The final amount after applying the discount.
    """
    discount = amount * 0.10
    final_amount = amount - discount
    return final_amount
print("Final Amount:", calculate_final_amount(1000))
print("Final Amount:", calculate_final_amount(2000))
```

Output:

```
Final Amount: 900.0
Final Amount: 1800.0
```

Explanation:
In the refactored code, we have defined a function called `calculate_final_amount` that takes an `amount` as an argument. This function calculates the discount by multiplying the amount by 0.10 (which represents a 10% discount) and then subtracts the discount from the original amount to get the final amount. The function is well-documented with a docstring that explains its purpose, its parameters, and its return value.

Task 3: Refactoring – Improving Loop and Conditional Performance

Sample input code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

Prompt: Enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Code:

```
def check_students(check_list, students):
    """Check if names in the check list exist in the students list.
    Args:
        check_list (list): A list of names to check.
        students (list): A list of student names.
    Returns:
        dict: A dictionary with names as keys and their existence status as values.
    """
    student_set = set(students)  # Convert students list to a set for O(1) lookups
    results = {}
    for name in check_list:
        results[name] = name in student_set  # Check existence in the set
    return results
check_list = ["Charan", "Esha", "Bhavya"]
students = ["Anil", "Bhavya", "Charan", "Divya"]
results = check_students(check_list, students)
for name, exists in results.items():
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

Output:

```
Charan found
Esha not found
Bhavya found
```

Explanation:

Before refactoring, the code uses nested loops to check for the existence of names, resulting in O(n*m) time complexity, where n is the length of the check list and m is the length of the students list.

After refactoring, the code converts the students list into a set, allowing for O (1) average time complexity for lookups, resulting in an overall O(n) time complexity for checking all names in the check list. This significantly improves performance, especially as the size of the students list increases.

In summary, the refactored code is more efficient and maintainable, as it eliminates the need for nested loops and reduces the time complexity from O(n*m) to O(n).

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Sample input code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Prompt: Improve maintainability by replacing hardcoded numerical values with named constants and define descriptive constants at the beginning of the program.

Code:

```
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000
RATE = 5
TIME = 2
def calculate_simple_interest(principal, rate, time):
    """Calculate the simple interest.
    Args:
        principal (float): The principal amount.
        rate (float): The annual interest rate (in percentage).
        time (float): The time the money is invested for (in years).
    Returns:
        float: The calculated simple interest.
    """
    return (principal * rate * time) / 100
print("Simple Interest:", calculate_simple_interest(PRINCIPAL_1, RATE, TIME))
print("Simple Interest:", calculate_simple_interest(PRINCIPAL_2, RATE, TIME))
```

Output:

```
Simple Interest: 100.0
Simple Interest: 200.0
```

Explanation:

By defining constants for the principal amounts, interest rate, and time, we improve the readability and maintainability of the code. The function `calculate_simple_interest` is reusable and can be easily modified if the formula for calculating interest changes in the future. This design allows for better organization and separation of concerns, making it easier to understand and maintain the code.

Task 5: Re factoring – Enhancing Variable Namingand Code Clarity

Sample input code:

```
x = 50
y = 30
z = x + y
print(z)
```

Prompt: Improve code readability by renaming unclear variables and adding meaningful comments.

Code:

```
# Define constants for the numbers being added
NUMBER_1 = 50
NUMBER_2 = 30
# Calculate the sum of the two numbers
def calculate_sum(num1, num2):
    """Calculate the sum of two numbers.
    Args:
        num1 (float): The first number.
        num2 (float): The second number.
    Returns:
        float: The sum of the two numbers.
    """
    return num1 + num2
# Calculate and print the result
result = calculate_sum(NUMBER_1, NUMBER_2)
print("The sum of", NUMBER_1, "and", NUMBER_2, "is:", result)
```

Output:

```
The sum of 50 and 30 is: 80
```

Explanation:

In the refactored code, defined constants for the numbers being added to improve readability and maintainability. And created a function `calculate_sum` that takes two numbers as arguments and returns their sum. This makes the code more reusable and easier to understand. Additionally, added docstrings to the function to explain its purpose, parameters, and return value. Finally, calculated the result using the function and printed it in a clear and descriptive manner.